2022 Paper to Code Mapping
Magnetic-Field-Inspired Navigation

2026-06-18

Purpose

This document compares the 2022 paper *Magnetic-Field-Inspired Navigation for Robots in Complex and Unknown Environments* with the current implementation in `src/mfinav/double_integrator.py`.

The goal is not to claim equivalence. The goal is to make the current status explicit:

- what the paper says,
- where that idea appears in the code,
- whether the implementation is exact, approximate, or missing.

Primary paper source: `https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2022.834177/full`

Legend

- **Exact**: the current code follows the same structure and intent closely enough to be treated as the same mechanism.
- **Approximate**: the current code is clearly inspired by the paper but does not yet implement the same equation or assumption literally.
- **Missing**: the paper concept is not yet represented faithfully in code.

Reference Code Locations

- Goal controller: lines 225–286
- Local sensing and closest-point averaging: lines 289–327
- Boundary-following field: lines 330–357
- Collision-avoidance field: lines 360–376
- Combined control law: lines 379–396
- Simulation loop and saturation assumptions: lines 567–608

High-Level Control Law

Paper Section 4 states that the obstacle term is split into a boundary-following vector field and a collision-avoidance vector field. Paper Section 5 states that the point robot is steered by control law (38), with a goal term and the obstacle term.

$$u = F_g + F_o \tag{1}$$

$$F_o = F_b + F_a \tag{2}$$

Current code

The current code follows exactly that high-level decomposition:

- `goal_cmd` in lines 392–393
- `boundary_cmd` in lines 394
- `collision_cmd` in line 395
- sum in line 396

Status: **Exact at architecture level**

Go-To-Goal Term F_g

Paper

Paper Section 5 says the authors first used PD control from Eq. (39) as the goal attraction term. The same section then says geometric control was introduced as an alternative because plain PD does not guarantee constant speed.

The PD form described in the paper is the standard point-robot attractive term:

$$F_g = -K_P(p - p_g) - K_D\dot{p} \tag{3}$$

Equivalently, with $e = p - p_g$,

$$F_g = -K_P e - K_D\dot{p} \tag{4}$$

Current code

The current code supports three goal modes:

- `goal_mode = "pd"` at lines 269–273 and 274–286
- `goal_mode = "geometric"` at lines 274–284
- `goal_mode = "hybrid"` at lines 274–286

In PD mode, the implemented term is:

$$u_g = k_{\text{scale}} K_P (p_g - p) - K_D \dot{p} \tag{5}$$

which is equivalent to the paper PD form when $k_{\text{scale}} = 1$.

In geometric mode, the current code uses:

$$u_g = -k K_{P,r} (\|v\| - v_{\max}) \hat{v} + K_\omega \|v\| \hat{n} (\hat{n}^\top \hat{g}) \tag{6}$$

In hybrid mode, the geometric-like term is used far from the goal and the PD term is used near the goal.

Code excerpt

```
if self.config.use_legacy_goal_relaxation:
    kp_scale = self._goal_weight(goal_vector, observation.closest_obstacle_vector)
else:
    kp_scale = 1.0

if self.config.goal_mode == "pd":
    return kp_scale * self.config.kp_goal * goal_vector - self.config.kd_goal * state.
    velocity

if self.config.goal_mode == "geometric":
    speed = _norm(state.velocity)
    if goal_mag > self.config.magni_bound:
        if speed > EPS:
            direction = _unit(state.velocity)
        else:
            direction = _unit(goal_vector)

        speed_error = -(speed - self.config.speed_limit)
        vel_attr = (self.config.kp_goal_relaxed * kp_scale) * speed_error * direction

        if speed > EPS:
            angle_error = _signed_angle(state.velocity, goal_vector)
            omega = self.config.kp_geom * kp_scale * angle_error
            force_add = omega * _perp_left(state.velocity)
        else:
            force_add = np.zeros(2, dtype=float)
        return vel_attr + force_add

    return kp_scale * self.config.kp_goal * goal_vector - self.config.kd_speed * state.
    velocity

return kp_scale * self.config.kp_goal * goal_vector - self.config.kd_speed * state.
    velocity
```

This is closer to the ROS-era SO(3) geometric steering than the earlier simplified lateral heuristic, but it is still a 2D analogue rather than a full matrix-logarithm reconstruction.

Comparison

Paper item	Code location	Status	Notes
PD goal term F_g from Eq. (39)	lines 269–273, 286	Approximate	The PD structure is now much closer, but the same controller class still supports non-paper modes and optional scaling logic.
Geometric control alternative	lines 274–284	Approximate	The 2D steering law is now closer to the ROS-era formulation, but it is still a 2D analogue rather than a full matrix-logarithm reconstruction.
Implementation choice "PD first, geometric alternative"	lines 200–223, 269–286	Approximate	The repo now exposes explicit paper PD and paper geometric modes, but the pragmatic default is still a hybrid controller not stated in the paper.

Boundary-Following Field F_b

Paper

Paper Section 4.1 says:

- F_b is induced by an artificial current on the obstacle surface,
- the artificial current l_o has the same direction as the projection of the robot motion direction l_a onto the obstacle surface,
- F_b is the term responsible for boundary following.

The paper also states in Lemma 1 that F_b does not affect robot speed.

Current code

Current code computes:

$$\hat{l}_a = \frac{v}{\|v\|} \tag{7}$$

$$\hat{r} = \frac{-r_o}{\|r_o\|} \tag{8}$$

$$l_o \approx \hat{l}_a - (\hat{l}_a^\top \hat{r})\hat{r} \tag{9}$$

with a perpendicular fallback when the projection becomes too small (lines 342–353).
Two field modes now exist:

- **paper mode** at lines 355–357:

$$F_b^{\text{code}} = \sigma_b c\, l_{a,\perp} \left(l_{a,\perp}^{\top} \left(\frac{\|v\|}{d} l_{o,\perp} \right) \right) \tag{10}$$

- **pragmatic mode** at lines 356–357:

$$F_b^{\text{code}} = \sigma_b c\, v_{\text{guide}} \frac{\hat{l}_o}{d} \tag{11}$$

Code excerpt

```
dist_to_obs = observation.used_obstacle_vector
dist_from_obs = -dist_to_obs
dist_from_obs_hat = _unit(dist_from_obs)

obs_cur = agent_cur - float(np.dot(agent_cur, dist_from_obs_hat)) * dist_from_obs_hat
obs_cur_mag = _norm(obs_cur)
if self.config.field_mode == "paper":
    if obs_cur_mag >= self.config.epsilon_current:
        obs_cur = obs_cur / obs_cur_mag
        self.previous_surface_current = obs_cur.copy()
    elif self.previous_surface_current is not None:
        obs_cur = self.previous_surface_current.copy()
    else:
        return np.zeros(2, dtype=float)

    velocity_3 = _embed_2d(state.velocity)
    agent_current_3 = _embed_2d(agent_cur)
    obs_current_3 = _embed_2d(obs_cur)
    b_field_3 = _skew3(obs_current_3) @ velocity_3 / max(distance, EPS)
    force_3 = self.config.c_field * (_skew3(agent_current_3) @ b_field_3)
    return _project_2d(force_3)
```

The pragmatic mode still uses the direct tangent-guidance surrogate:

```
if obs_cur_mag < self.config.epsilon_current:
    obs_cur = _perp_left(dist_from_obs_hat)
else:
    obs_cur = obs_cur / obs_cur_mag

sigma_b = max(0.0, min(1.0, (self.config.r_l - distance) / max(self.config.r_l - self.
    config.r_la, EPS)))
guidance_speed = max(speed, 0.5 * self.config.speed_limit)
return sigma_b * self.config.c_field * guidance_speed * obs_cur / max(distance, EPS)
```

Comparison

Paper item	Code location	Status	Notes

Artificial current as projection of robot motion onto obstacle surface	lines 342–353	Exact in spirit	This part is structurally aligned with the paper.
Magnetic boundary-following field derived from that artificial current	lines 355–357	Exact relative to the current 2D derivation target	The paper-mode implementation now directly matches the explicit 2D target $F_b^{2D} = -c\frac{l_o \times_2 v}{\|r_o\|} J l_a$.
Property that F_b does not affect speed	lines 355–357	Exact relative to the 2D target field	The paper-mode F_b is explicitly orthogonal to l_a. Any remaining speed-assumption mismatch now comes from other parts of the simulator or controller, not from the F_b formula itself.

Collision-Avoidance Field F_a

Paper

Paper Section 4.2 states that F_a is needed for concave obstacles because boundary following alone does not regulate the distance to the obstacle surface. Lemma 5 states that F_a also does not affect speed.

Current code

Current code uses a short-range radial term whose component along the motion direction is removed:

$$F_a^{\text{code}} = -\sigma_a c_\perp \left(\frac{1}{d} - \frac{1}{r_{la}}\right) \frac{r_o}{d} \tag{12}$$

In `field_mode = "paper"`, this is the primary F_a law. This matches the old ROS `field=2` structure better than the previously tried $l_o^\perp$-based magnetic 2D target.

Code excerpt

```
if self.config.field_mode == "paper":
    repulsion = -self.config.c_perp * (1.0 / max(distance, EPS) - 1.0 / self.config.r_la)
     * (
        dist_to_obs / max(distance, EPS)
    )
    if _norm(agent_cur) > EPS:
        repulsion = repulsion - float(np.dot(repulsion, agent_cur)) * agent_cur
    return repulsion

sigma_a = max(0.0, min(1.0, (self.config.r_la - distance) / max(self.config.r_la, EPS)))
repulsion = -sigma_a * self.config.c_perp * (1.0 / max(distance, EPS) - 1.0 / self.config
    .r_la) * (
```

```
    dist_to_obs / max(distance, EPS)
)
if _norm(agent_cur) > EPS:
    repulsion = repulsion - float(np.dot(repulsion, agent_cur)) * agent_cur
return repulsion
```

Comparison

Paper item	Code location	Status	Notes
Repulsive collision-avoidance field for concave cases	lines 364–376	Exact in purpose	The code clearly contains the intended mechanism.
Exact paper field expression for F_a	lines 372–375	Attempted 2D target match, but still not validated	The implementation was updated to the current explicit 2D target, but the resulting benchmark degradation strongly suggests that the present 2D target for F_a is not yet the correct reduction.
Property that F_a does not affect speed	lines 372–375	Intended by construction, but behavior still problematic	The current 2D target keeps F_a orthogonal to l_a by construction, but the overall benchmark behavior shows that preserving orthogonality alone is not enough to claim a correct paper reduction.

Thresholded Use of r_l and r_{la}

Paper

Paper Section 5 states:

- F_b is only used when the closest distance falls below r_l,
- F_a is only used when the robot gets closer still, below r_{la}.

Current code

- $F_b = 0$ if $d > r_l$ at lines 335–337
- $F_a = 0$ if $d > r_{la}$ at lines 365–367
- smooth activation weights σ_b and σ_a are used at lines 355 and 372

Status: **Approximate**

The threshold logic is correct in spirit. The smooth activation is an additional implementation choice that is not stated in the paper excerpt.

Closest-Point Detection

Paper

Paper Section 4.4 says that the original closest-point detection chooses the smallest sensed distance from current sensor readings.

Current code

Current code does exactly that first:

- compute all sensed vectors: lines 294–301
- compute distances: line 302
- choose smallest distance: lines 303–305

Status: **Exact**

Code excerpt

```
if hasattr(obstacle, "sensed_vectors"):
    sensed_vectors = obstacle.sensed_vectors(
        state.position,
        self.config.sensing_samples_per_obstacle,
        self.config.sensing_angular_window,
    )
else:
    sensed_vectors = [obstacle.closest_vector(state.position)]
distances = [_norm(vec) for vec in sensed_vectors]
min_index = int(np.argmin(distances))
dist_to_obs = sensed_vectors[min_index]
min_distance = distances[min_index]
```

Closest-Point Averaging for Non-Unique Closest Points

Paper

Paper Section 4.4 states:

- select several sensed distances smaller than threshold δr,
- average them,
- for concave/non-unique cases, use the average if it is smaller in magnitude than the single closest sensed position,
- for convex cases, revert to the standard closest point.

Current code

The current code follows exactly that logic:

- collect all vectors within the δr band: lines 307–309
- average them: line 310
- if the average is shorter, use it: lines 315–318
- otherwise use the single closest vector: lines 317–318

Status: **Exact in logic**

Code excerpt

```
close_vectors = [
    vec for vec, distance in zip(sensed_vectors, distances) if distance <= min_distance +
     self.config.delta_r
]
averaged_vector = np.mean(np.array(close_vectors, dtype=float), axis=0)

if _norm(averaged_vector) < min_distance:
    used_vector = averaged_vector
else:
    used_vector = dist_to_obs
```

Goal Relaxation

Paper

Paper Section 4.4 says goal relaxation is introduced for large obstacles where following the boundary may increase the distance to the goal beyond the initial goal distance. The paper description is qualitative in the Frontiers article:

- decrease F_g when the robot is close to the obstacle and the goal is still occluded,
- increase F_g when the obstacle no longer blocks the goal.

The paper explicitly points back to earlier work for details.

Current code

The current implementation contains a legacy goal-relaxation weight:

$$k_{\mathrm{GR}} = \left(1 - e^{-d_o/1.5}\right)(1 - \cos\theta)\exp\left(-\frac{d_g - d_{g,0}}{0.1}\right)\exp\left(-\frac{d_g - d_{g,\min}}{0.1}\right) \tag{13}$$

This logic appears in lines 231–256 and is only active when `use_legacy_goal_relaxation = True` (line 269).

Code excerpt

```
goal_mag = _norm(goal_vector)
obs_mag = _norm(obs_vector)
if goal_mag < EPS or obs_mag >= self.config.r_l or obs_mag < EPS:
    self.min_distance_to_goal = min(self.min_distance_to_goal, goal_mag)
    return 1.0

if self.initial_goal_distance is None:
    self.initial_goal_distance = goal_mag
self.min_distance_to_goal = min(self.min_distance_to_goal, goal_mag)

cos_angle = float(np.dot(goal_vector, obs_vector) / max(goal_mag * obs_mag, EPS))
cos_angle = max(-1.0, min(1.0, cos_angle))
weight = 1.0 - cos_angle

if self.initial_goal_distance is None or goal_mag <= self.initial_goal_distance:
    new_weight = 1.0
else:
    new_weight = math.exp(-(goal_mag - self.initial_goal_distance) / 0.1)

weight_exit = math.exp(-(goal_mag - self.min_distance_to_goal) / 0.1)
distance_scale = 1.0 - math.exp(-obs_mag / 1.5)
return distance_scale * weight * new_weight * weight_exit
```

Comparison

Status: **Approximate / inherited from earlier ROS work**

This is not a direct equation-level implementation from the 2022 Frontiers paper. It is a pragmatic reuse of an earlier goal-relaxation mechanism from the legacy codebase. However, unlike earlier revisions of this clean repository, the paper-aligned modes now enable this mechanism instead of bypassing it.

Local Sensing Assumption

Paper

The paper repeatedly states that only local sensory information is assumed, and that no prior map is required.

Current code

The current code honors that spirit through local obstacle samples. In the paper-aligned modes, these samples now come from ray-based sensing rather than direct boundary sampling:

- rays are cast around the robot over a full angular sweep
- the nearest obstacle intersection on each ray is selected
- only the currently visible local obstacle points are passed to the closest-point and averaging logic

Status: **Closer, but still approximate**

This is much closer to the paper assumption than the earlier direct analytic boundary sampling, but it is still not the same as the ROS range-sensor or point-cloud pipeline used in the paper experiments.

Dynamics and Actuation Assumptions

Paper

The paper explicitly assumes non-saturatable actuators.

Current code

The pragmatic benchmark mode clips both acceleration and speed:

- acceleration clipping: lines 581–582
- speed clipping: lines 599–600

Status: **Matched for paper-aligned modes, different for pragmatic mode**

The paper-aligned modes now run without acceleration or speed clipping. The pragmatic mode still clips both for robustness.

Point-Robot Parameter Table

Paper Table 1 gives the following point-robot parameters:

- $c = 10$
- $c_\perp = 20$
- $K_P = 0.04$
- $K_D = 0.5$
- $r_l = 3$
- $r_{la} = 2$
- $\epsilon = 3 \times 10^{-6}$

The code paths intended to mirror these values are:

- `make_paper_pd_config()` and `make_paper_geometric_config()`

Status: **Exact for the numeric table, not yet exact for the full behaviour**

Not Exact Yet

The remaining non-exact parts are:

1. **Boundary-following magnetic field** F_b. The 2D paper-mode implementation now matches the explicit 2D derivation target. The remaining open question is no longer the 2D formula itself, but whether that 2D target is the exact intended reduction of the paper's 3D field law.

2. **Collision-avoidance field** F_a. The implementation has now been matched to the *current* explicit 2D target, but the resulting behaviour degrades strongly. This indicates that the current 2D target for F_a is itself still likely wrong or incomplete.

3. **Geometric controller**. The current version is a 2D analogue of the ROS-era SO(3)-style steering logic, not a full 3D matrix-logarithm reconstruction of the original idea.

4. **Goal relaxation**. The current GR term is no longer bypassed in paper mode and now has a distinct paper-style path, but it is still not rebuilt directly from the 2022 paper as a verified equation-level reconstruction.

5. **Local sensing model**. The paper-aligned modes now use ray-based local sensing, which is much closer to the paper assumption. However, it is still not the same as the ROS sensing pipeline used in the paper experiments.

6. **Actuation assumptions**. These are now matched for the paper-aligned modes, but not for the pragmatic benchmark mode.

7. **Non-convex benchmark behaviour**. After matching these assumptions, the paper-style PD and geometric modes are both collision-free in the U-shaped polygon benchmark, and the geometric mode gets much closer to the goal. However, neither mode fully converges there. This suggests the remaining mismatch is now more concentrated in the exact controller equations and benchmark abstraction.

Overall Assessment

Component	**Status**	**Comment**
High-level split $u = F_g + F_b + F_a$	Exact	Architecture matches the paper.
Closest-point detection	Exact	Single minimum-distance reading is implemented directly.
Section 4.4 closest-point averaging	Exact in logic	One of the strongest matches in the current code.
PD goal attraction	Approximate	Much closer now, but still implemented inside a multi-mode controller with optional non-paper logic.

Geometric control path	Closer, with improved non-convex behavior	The controller is still a 2D analogue rather than a line-by-line verified reconstruction, but low-speed turning and near-goal velocity tracking now behave much better in the polygon benchmarks, including the non-convex U-shape case.
Boundary-following magnetic field F_b	Exact relative to the 2D target	The current paper mode now matches the explicit 2D derivation target directly.
Collision-avoidance field F_a	Closer, but still approximate	The current code now follows the ROS-backed short-range projected repulsion target again; this is much more plausible than the rejected magnetic $l_o^{\perp}$ target, but it is still not a fully verified equation-level reconstruction from the paper text alone.
Goal relaxation	Approximate	Paper-aligned modes now use a dedicated paper-style GR path, but it is still not a verified equation-level reconstruction of the 2022 paper.
Local sensing assumption	Closer, but still approximate	Paper-aligned modes now use ray-based local sensing, but not the same sensing pipeline as the ROS work.
Actuation assumptions	Matched in paper modes, different in pragmatic mode	Paper-style presets now avoid clipping; pragmatic mode still clips for robustness.

Practical Conclusion

The current implementation is closest to the paper in:

- overall architecture,
- thresholded obstacle-field activation,
- closest-point and closest-point-averaging logic.

The current implementation is farthest from the paper in:

- the exact magnetic-field equations for F_b and F_a,
- the exact goal-control choice used in each paper experiment,

- the exact remaining sensing and non-convex benchmark abstractions.

Therefore, the current repository should still be described as a **paper-inspired clean reimplementation**, not yet a verified reproduction of the 2022 point-robot algorithm.